CRUZR S2 · VISIÓN · MANOS V4

Plan de trabajo: reconocimiento y manipulación de una botella con Cruzr S2 y manos v4

Plan de desarrollo: detección de botella, capturas RGB-D, calibración, planificación y agarre adaptativo.

Plan de trabajo · 15 de octubre de 2026
Integración y validación física pendientes

Contenido

1. Objetivo y alcance de la primera versión
    1.1 Resultado funcional esperado
2. Estado de partida: hechos comprobados y limitaciones
3. Organización, estados y reglas de ejecución
    3.1 Significado de las etiquetas
    3.2 Roles
    3.3 Puertas de avance
4. Cantidad de posiciones, capturas y ensayos
    4.1 Campaña inicial de evaluación, sin entrenamiento
    4.2 Registro por escena
5. Fase 0 — entorno de trabajo e inventario
    5.1 Directorio de datos fuera de Git — PC
    5.2 Red — LECTURA desde PC
    5.3 Contenedores — LECTURA
    5.4 Inspección de cámaras — LECTURA en Vision
6. Fase 1 — capturas RGB-D estáticas
    6.1 Crear el capturador de desarrollo — PC
    6.2 Crear un lanzador de copia/captura — PC
    6.3 Completar la cuadrícula
7. Fase 2 — detector preentrenado COCO en el PC
    7.1 Elección inicial y condiciones
    7.2 Crear una imagen Docker local — PC, no robot
    7.3 Programa de inferencia 2D — PC
    7.4 Decisión después de la primera prueba
    7.5 Criterios iniciales de evaluación visual
8. Fase 3 condicional — entrenamiento si el modelo preentrenado no basta
    8.1 Datos y partición
    8.2 Anotación y formato
    8.3 Entrenar y evaluar — PC
9. Fase 4 — profundidad, registro y posición 3D
    9.1 Escala y calidad de profundidad
    9.2 Alineación RGB-D y distorsión
    9.3 Primer cálculo: punto superficial, no centro ni pose de agarre
    9.4 Casos de fallo de profundidad
10. Fase 5 — coordenadas del robot y calibración del TCP
    10.1 Convenciones que deben fijarse
    10.2 Inspeccionar TF — LECTURA
    10.3 Calibración cámara–base
    10.4 TCP y geometría de la mano v4
11. Fase 6 — adaptación del movimiento y planificación
    11.1 Qué se conserva de la demostración PICO
    11.2 Waypoints relativos a la botella
    11.3 Resolución y admisión
    11.4 Comandos disponibles para descubrir el SDK — LECTURA en Motion
    11.5 Componentes que aún deben desarrollarse
    11.6 Pruebas de software antes de movimiento
12. Fase 7 — prensión, fuerza y primera prueba física
    12.1 Lo conocido sobre el perfil actual
    12.2 Información necesaria antes de agarrar
    12.3 Secuencia física gradual
    12.4 Máquina de estados y errores
13. Fase 8 — demostraciones y grabación completa
    13.1 Movimiento y dedos — herramientas existentes
    13.2 Añadir visión para un futuro dataset
14. Fase 9 — validación del sistema completo
    14.1 Tabla de ensayo
    14.2 Qué significa éxito
    14.3 Si el ensayo falla
15. Entregables, cierre y mantenimiento
16. Orden práctico de las siguientes sesiones
17. Referencias y límites de la documentación

ID: BOTELLA-VISION-GRASP-20261015
Fecha: 15 de octubre de 2026. Zona: Europe/Madrid.
Estado: PLAN DE DESARROLLO. Percepción de botella y agarre adaptativo pendientes de implementación y validación.
Destinatarios: responsable técnico, programador de integración y operador del robot.
Repositorio: `Humanoide`. Rama al redactar: `feat/pico-v4-workflows-and-scenario-fixes`.

1. Objetivo y alcance de la primera versión

Lograr que el robot identifique una botella, determine dónde está sobre una mesa, calcule un acercamiento de una mano, la sujete y la deposite en un destino conocido. La posición de origen podrá variar dentro de una zona pequeña previamente validada.

La primera versión tendrá estas condiciones:

- Una botella conocida, ligera, estable, de pie y preferentemente opaca.
- Una sola botella candidata en la zona de trabajo.
- Mesa, robot y cámara de cintura inmóviles durante la observación y planificación.
- Un único brazo elegido para la tarea; la otra mano permanece en postura revisada.
- Destino B fijo y despejado. La botella permanece quieta durante el acercamiento.
- Percepción y planificación tradicionales; el VLA no interviene en el movimiento.
- Supervisión física en las primeras pruebas. La integración no se considera autónoma industrial por completar una demostración.

Las versiones con botellas diferentes, objetos transparentes, botellas tumbadas, objetos en movimiento, selección entre varias botellas y ajuste aprendido del agarre se abordarán después y con nuevas pruebas.

1.1 Resultado funcional esperado

```
Imagen + profundidad + calibración + estado actual
                  ↓
Detección/segmentación de botella
                  ↓
Localización 3D y comprobación de calidad
                  ↓
Transformación al sistema de coordenadas del robot
                  ↓
Construcción del marco de agarre y planificación de la mano
                  ↓
Validación de alcance, colisiones, continuidad y condiciones actuales
                  ↓
Acercar → cerrar → comprobar → elevar → depositar → abrir → retirar
```

Reconocer la categoría «botella» no determina por sí solo su pose 3D, el punto de agarre, la fuerza necesaria ni una trayectoria libre de colisiones.

2. Estado de partida: hechos comprobados y limitaciones

Elemento	Estado al redactar	Consecuencia
Manos v4 y perfil SPS	Instalados y usados en esta unidad	Redescubrir hardware y contenedores después de cualquier actualización

Elemento	Estado al redactar	Consecuencia
PICO v4 mediante mandos	Probado en vacío	Grips para brazos; gatillos para manos; `Hands` óptico no es necesario para este flujo
Captura de posiciones reales y órdenes de manos	Probada con ROS bag	Usar el grabador completo; el `.motion` nativo solo no recogió el cierre en modo corriente
Reproducción geométrica de una toma	Probada en vacío, a mitad de velocidad	No demuestra reproducción de esfuerzo ni retención de una botella
Detector suministrado `workbin`	Especializado en cajas	No reutilizar sus dimensiones o límites como contrato de agarre de botellas
Detector COCO de botellas	Disponible como opción externa; no instalado por este plan	Probar en el PC con imágenes reales antes de entrenar
Localización métrica de la botella	Pendiente	La caja 2D del detector no autoriza movimiento
Calibración cámara–base–TCP de la mano elegida	Pendiente de validar para esta tarea	No copiar el TCP de las abrazaderas ni una transformación de otra unidad
Regulación mínima de fuerza y detección fiable de deslizamiento	No demostradas	No presentarlas como capacidades existentes
Agarre autónomo A→B de botella	Pendiente	Requiere un ejecutor específico, además de percepción

Lecturas realizadas para redactar este documento, sin movimiento:

- Motion y Vision son accesibles; se redescubrieron sus contenedores.
- El contenedor ROS de Vision dispone de `rclpy`, `rosbag2_py`, `cv_bridge`, OpenCV 4.5.4 y NumPy 1.21.5. No se instaló software en el robot.
- Cámara RGB-D de cintura: imagen `640×360`; color `rgb8`; profundidad `16UC1`.
- Sus imágenes usan `shm_msgs/msg/Image1m`, con un buffer de capacidad fija.
- `frame_id` y `encoding` son `shm_msgs/String`: se decodifican usando `size`. Leer todo el buffer como cadena introduce bytes residuales y nombres incorrectos.
- Ambas imágenes declaran `waist_front_rgbd_color_optical_frame`.
- Los dos `CameraInfo` tienen publicación `RELIABLE / TRANSIENT_LOCAL`. Una suscripción volátil no recuperó la calibración; la transiente sí.
- Color y profundidad anuncian los mismos intrínsecos principales, pero el color tiene distorsión no nula y la profundidad declara coeficientes nulos. **Esto obliga a verificar registro y rectificación antes de cruzar sus píxeles.**

Estas son observaciones históricas, no una garantía de disponibilidad mañana. Evidencia privada: `../Humanoide-vla-evidence/20261015_BOTTLE_PLAN/`.

3. Organización, estados y reglas de ejecución

3.1 Significado de las etiquetas

- **LECTURA:** consulta o suscripción; no manda trayectorias.
- **PC:** crea archivos, instala dependencias o ejecuta inferencia solo en el PC.
- **CAPTURA:** suscripciones pasivas que escriben datos.

- **FÍSICO:** puede mover el robot; requiere comprobaciones físicas y técnicas actuales.
- **PROPUESTA:** diseño o umbral de laboratorio; no es una capacidad certificada.
- **PENDIENTE DE IMPLEMENTAR:** no existe todavía una orden ejecutable validada.

Los bloques de código nuevos de esta guía son recetas de desarrollo. Se han revisado sintácticamente; no se ha realizado una instalación de YOLO ni un ensayo de agarre al redactarla. El capturador de imágenes incluido es un prototipo, que se debe verificar primero con una captura estática.

3.2 Roles

Rol	Responsabilidad
Responsable técnico	Aprobar geometría, calibraciones, TCP, límites y resultados de cada fase
Programador	Implementar herramientas, pruebas, registros y tratamiento de errores
Operador PICO/robot	Preparar la escena y ejecutar únicamente la fase acordada
Persona junto al paro	Vigilar el recorrido y actuar ante una emergencia

Antes de cada movimiento: manos instaladas y objeto libre/sujeto/apoyado bien identificado, postura medida, batería y cargador, ambos paros, bloqueo de ruedas, modo de trabajo, zona despejada, otros mandos detenidos y persona junto al paro. Con Ethernet conectado, el MVP mantiene el chasis inmóvil.

No combinar PICO, UI web y ejecutor automático como mandos simultáneos. Los grabadores pasivos sí pueden acompañar al único mando activo. No desactivar límites, watchdogs, anticolisión o protecciones para hacer pasar una prueba. Ante una interrupción con botella sujeta, no ejecutar HOME ni reiniciar el ciclo. Aplicar la guía de recuperación.

3.3 Puertas de avance

Puerta	Evidencia necesaria para avanzar	Si no pasa
G0: infraestructura	Ruta correcta, cámaras y tipos identificados, archivos de inventario	Resolver enlace/servicios, sin reiniciar controles como ensayo
G1: captura	RGB legible, profundidad sin corrupción, tiempos y metadatos conservados	Corregir QoS, formato, sincronía o almacenamiento
G2: reconocimiento	Detección de una botella en escenas positivas y rechazo de negativas	Mejorar escena/modelo o seguir rama de entrenamiento
G3: métrica	Registro RGB-D, escala de profundidad e intrínsecos verificados	No producir un destino físico desde esos datos
G4: coordenadas	Transformaciones cámara→base y base→TCP verificadas	Calibrar; no inventar ni publicar una TF para ocultar el problema
G5: plan	IK, límites, colisiones, continuidad y aproximación revisados	Replantear postura/zona/TCP o pedir soporte del SDK
G6: ensayo en vacío	Acercamiento reducido y repetible, sin objeto dentro del recorrido	Revisar geometría y planificación antes de agarrar
G7: agarre	Sujeción y depósito comprobados con botella ligera	Corregir prensión; no aumentar esfuerzo a ciegas

Puerta	Evidencia necesaria para avanzar	Si no pasa
G8: generalización	Pruebas en posiciones no usadas para ajustar parámetros	Ampliar datos y validar de nuevo

4. Cantidad de posiciones, capturas y ensayos

4.1 Campaña inicial de evaluación, sin entrenamiento

Propuesta para **una botella**:

Conjunto	Configuraciones	Capturas por configuración	Total
Desarrollo visual	9 posiciones × 3 giros	5	135 RGB-D
Escenas negativas	10 escenas sin botella objetivo	1	10 RGB-D
Prueba reservada	4 posiciones nuevas × 3 giros	5	60 RGB-D

Total inicial propuesto: **205 pares RGB-D**, cada uno con calibración y tiempos. Los primeros 145 corresponden a la campaña ya propuesta; las 60 capturas finales son un conjunto reservado, que no se usará para ajustar confianza o geometría.

Las 5 capturas de una configuración comprueban estabilidad y calidad, pero están correlacionadas: no son 5 escenas independientes ni aumentan por sí solas la diversidad. Los 135 ejemplos **no constituyen un mínimo garantizado para entrenar**. COCO ya incluye `bottle`; primero se evaluará un modelo preentrenado [R1, R2].

Marcar sobre la mesa una cuadrícula de 3×3. Definir su separación después de verificar alcance y visibilidad; no imponer centímetros universales. Identificar las posiciones `p01` a `p09`, y los giros `r000`, `r120`, `r240`. La botella sigue de pie. El giro sirve para variar etiqueta/apariencia; no presupone que el agarre cilíndrico necesite estimar yaw con precisión.

Reservar otras cuatro posiciones para prueba y registrar sus coordenadas reales. No incluir posiciones fuera del alcance para completar artificialmente la cuadrícula.

4.2 Registro por escena

Guardar: ID de botella, material, altura/diámetro medidos, masa aproximada, posición real de referencia y su incertidumbre, giro, iluminación, fondo, contenido/vacía, estado de la mano, cámara usada y postura del robot. Para medir error de pose, la referencia debe corresponder a un punto físico bien definido: por ejemplo, centro del eje a una altura marcada. Una marca en la mesa sin transformación a `base_link` no es una referencia 3D completa.

5. Fase 0 — entorno de trabajo e inventario

5.1 Directorio de datos fuera de Git — PC

```
cd /home/lacuna/proyectos/Robots/Humanoide
export BOTELLA_REPO="$PWD"
export BOTELLA_ROOT="$HOME/robot-data/botella-20261015"
mkdir -p "$BOTELLA_ROOT"/{tools,raw,results,models,manifests,calibration,dataset}
printf 'export BOTELLA_REPO=%q\nexport BOTELLA_ROOT=%q\n' \
  "$BOTELLA_REPO" "$BOTELLA_ROOT" > "$BOTELLA_ROOT/session.env"
git status --short --branch > "$BOTELLA_ROOT/manifests/git-status.txt"
git rev-parse HEAD > "$BOTELLA_ROOT/manifests/git-head.txt"
```

En otra terminal, cargar el mismo entorno:

```
source "$HOME/robot-data/botella-20261015/session.env"
```

No modificar `HOME`. No añadir pesos, imágenes, bags, contraseñas o backups al repositorio. Si el directorio ya contiene una campaña, usar otro nombre y conservar los datos anteriores.

5.2 Red — LECTURA desde PC

```
ip -4 -o addr show dev eno1
ip route get 192.168.11.2
ip route get 192.168.11.3
cat /sys/class/net/eno1/carrier
ping -c 3 -W 1 192.168.11.2
ping -c 3 -W 1 192.168.11.3
```

Para la instalación conocida: `eno1`, origen `192.168.11.250`, enlace activo. Si aparece Wi-Fi o no hay carrier, resolver el enlace antes de un ensayo físico. No desactivar la conexión de Internet del PC para corregir la ruta al robot.

5.3 Contenedores — LECTURA

```
ssh walker@192.168.11.2 'docker ps --format "{{.Names}}  {{.Image}}  {{.Status}}"'
ssh walker@192.168.11.3 'docker ps --format "{{.Names}}  {{.Image}}  {{.Status}}"'
```

Valores observados, que deben contrastarse con la salida:

```
export BOTELLA_ROS_VISION=walker-ros.ros2-1
export BOTELLA_ROS_MOTION=walker-ros.ros2-1
export BOTELLA_APP_MOTION=walker-motion.manipulation_robot_app-1
```

Los nombres ROS son iguales en dos hosts distintos. No confundir Vision `.3` con Motion `.2`. Si cambian, actualizar las variables y documentarlo. No recrear contenedores, cambiar `HW_TYPE` o volver a instalar el bus de manos por seguir este plan.

5.4 Inspección de cámaras — LECTURA en Vision

```
ssh -t walker@192.168.11.3 \
  "docker exec -it $BOTELLA_ROS_VISION bash"
```

Dentro del contenedor:

```
source /opt/ros/humble/setup.bash
export ROS2CLI_DISABLE_DAEMON=1
timeout 8 ros2 topic list -t
timeout 8 ros2 interface show shm_msgs/msg/Image1m
timeout 8 ros2 topic info -v /sensor/camera/waist_front_rgbd/color/raw
timeout 8 ros2 topic info -v /sensor/camera/waist_front_rgbd/depth/raw
timeout 8 ros2 topic info -v /sensor/camera/waist_front_rgbd/color/info
```

Cámara inicial propuesta:

Dato	Topic observado	Tipo
RGB	`/sensor/camera/waist_front_rgbd/color/raw`	`shm_msgs/msg/Image1m`
Profundidad	`/sensor/camera/waist_front_rgbd/depth/raw`	`shm_msgs/msg/Image1m`
Intrínsecos RGB	`/sensor/camera/waist_front_rgbd/color/info`	`sensor_msgs/msg/CameraInfo`
Intrínsecos profundidad	`/sensor/camera/waist_front_rgbd/depth/info`	`sensor_msgs/msg/CameraInfo`

Comprobar calibración latcheada:

```
timeout 8 ros2 topic echo --once --no-daemon \
  --qos-reliability reliable --qos-durability transient_local \
  /sensor/camera/waist_front_rgbd/color/info sensor_msgs/msg/CameraInfo

timeout 8 ros2 topic echo --once --no-daemon \
  --qos-reliability reliable --qos-durability transient_local \
  /sensor/camera/waist_front_rgbd/depth/info sensor_msgs/msg/CameraInfo
```

Un timeout consultando un `CameraInfo` con QoS volátil no demuestra ausencia de calibración. Consultar primero el QoS anunciado. Si no existe publisher, no insistir cambiando parámetros del controlador; revisar el servicio y la calibración con el responsable técnico.

Para observar frecuencia durante unos segundos:

```
timeout 6 ros2 topic hz /sensor/camera/waist_front_rgbd/color/raw
timeout 6 ros2 topic hz /sensor/camera/waist_front_rgbd/depth/raw
```

El código 124 de `timeout` significa que se agotó el intervalo; si se mostraron frecuencias, no implica por sí solo un fallo de cámara. No encadenar estas consultas en un script que interprete cualquier 124 como pérdida del sensor.

Salir del contenedor y del SSH cuando termine la inspección:

```
exit
exit
```

Éxito: tipos y publishers identificados, cámara visible, calibración obtenida. **Fallo:** volver a red, contenedor correcto, QoS y servicio de cámara. No avanzar a 3D si faltan intrínsecos o no se conoce la codificación/unidad de profundidad.

6. Fase 1 — capturas RGB-D estáticas

6.1 Crear el capturador de desarrollo — PC

Este programa solo se suscribe. No crea publicadores, clientes de acciones ni servicios de movimiento. Es compatible con las cadenas y buffers SHM observados. Guarda RGB como PNG, profundidad nativa como NumPy y metadatos JSON.

```
cat > "$BOTELLA_ROOT/tools/capture_rgbd.py" <<'PY'
import argparse
from collections import deque
import json
from pathlib import Path
import time
import cv2
import numpy as np
import rclpy
from rclpy.qos import QoSProfile, ReliabilityPolicy, DurabilityPolicy
from rosidl_runtime_py.utilities import get_message
from rosidl_runtime_py.convert import message_to_ordereddict
from sensor_msgs.msg import CameraInfo


def text(value):
    if isinstance(value, str):
        return value
    # size excludes residual bytes in the fixed SHM array.
    return bytes(value.data[:value.size]).decode('utf-8', errors='strict')


def stamp(msg):
    return msg.header.stamp.sec * 1_000_000_000 + msg.header.stamp.nanosec


def array(msg):
    h, w, step = int(msg.height), int(msg.width), int(msg.step)
    encoding = text(msg.encoding)
    needed = h * step
    if min(h, w, step) <= 0 or needed > len(msg.data):
        raise ValueError('Invalid SHM dimensions/buffer')
    data = bytes(msg.data[:needed])
    if encoding in ('rgb8', 'bgr8'):
        if step < w * 3:
            raise ValueError('Invalid color stride')
        image = np.ndarray((h, w, 3), np.uint8, buffer=data,
                           strides=(step, 3, 1)).copy()
        return cv2.cvtColor(image, cv2.COLOR_RGB2BGR) if encoding == 'rgb8' else image
    if encoding in ('16UC1', '32FC1'):
        size = 2 if encoding == '16UC1' else 4
        if step < w * size:
            raise ValueError('Invalid depth stride')
        dtype = np.dtype(('>' if msg.is_bigendian else '<') +
                         ('u2' if size == 2 else 'f4'))
        return np.ndarray((h, w), dtype, buffer=data,
                          strides=(step, size)).copy()
    raise ValueError('Unsupported encoding: ' + encoding)


def main():
    p = argparse.ArgumentParser()
    p.add_argument('--out', type=Path, required=True)
    p.add_argument('--count', type=int, default=5)
    p.add_argument('--max-dt-ms', type=float, default=50.)
    p.add_argument('--require-info', action='store_true')
    args = p.parse_args()
    if not 1 <= args.count <= 100 or not 0 < args.max_dt_ms <= 100:
        p.error('Invalid count/time tolerance')
    args.out.mkdir(parents=True, exist_ok=False)
    prefix = '/sensor/camera/waist_front_rgbd/'
    rclpy.init()
    node = rclpy.create_node('bottle_rgbd_passive_capture')
    buffers = {'color': deque(maxlen=20), 'depth': deque(maxlen=20)}
    infos = {}
    subscriptions = []
```

```
    deadline = time.monotonic() + 3
    while time.monotonic() < deadline:
        rclpy.spin_once(node, timeout_sec=.1)
    types = dict(node.get_topic_names_and_types())
    image_qos = QoSProfile(depth=5, reliability=ReliabilityPolicy.BEST_EFFORT,
                           durability=DurabilityPolicy.VOLATILE)
    info_qos = QoSProfile(depth=1, reliability=ReliabilityPolicy.RELIABLE,
                          durability=DurabilityPolicy.TRANSIENT_LOCAL)
    for kind in ('color', 'depth'):
        topic = prefix + kind + '/raw'
        if topic not in types or len(types[topic]) != 1:
            raise ValueError('Missing/ambiguous image type: ' + topic)
        subscriptions.append(node.create_subscription(
            get_message(types[topic][0]), topic,
            lambda msg, kind=kind: buffers[kind].append(msg), image_qos))
        subscriptions.append(node.create_subscription(
            CameraInfo, prefix + kind + '/info',
            lambda msg, kind=kind: infos.update({kind: msg}), info_qos))
    saved, last_rgb, last_depth, next_save = 0, 0, 0, 0.
    deadline = time.monotonic() + 35
    try:
        while saved < args.count and time.monotonic() < deadline:
            rclpy.spin_once(node, timeout_sec=.05)
            if not all(buffers.values()) or time.monotonic() < next_save:
                continue
            if args.require_info and len(infos) != 2:
                continue
            pairs = [(abs(stamp(c)-stamp(d)), c, d)
                     for c in buffers['color'] for d in buffers['depth']
                     if stamp(c) > last_rgb and stamp(d) > last_depth]
            if not pairs:
                continue
            dt, color, depth = min(pairs, key=lambda row: row[0])
            if dt > args.max_dt_ms * 1e6:
                continue
            now_ns = node.get_clock().now().nanoseconds
            ages = [now_ns-stamp(msg) for msg in (color, depth)]
            if any(age < -50_000_000 or age > 1_000_000_000 for age in ages):
                continue
            rgb, z = array(color), array(depth)
            if rgb.ndim != 3 or z.ndim != 2:
                raise ValueError('Unexpected image dimensions')
            folder = args.out / ('sample_%02d' % saved)
            folder.mkdir()
            if not cv2.imwrite(str(folder/'rgb.png'), rgb):
                raise IOError('Could not write RGB')
            np.save(folder/'depth_native.npy', z)
            meta = {
                'rgb_topic': prefix+'color/raw', 'depth_topic': prefix+'depth/raw',
                'rgb_stamp_ns': stamp(color), 'depth_stamp_ns': stamp(depth),
                'pair_dt_ms': dt/1e6, 'rgb_frame': text(color.header.frame_id),
                'depth_frame': text(depth.header.frame_id),
                'rgb_encoding': text(color.encoding), 'depth_encoding': text(depth.encoding),
                'rgb_size': [color.width, color.height], 'depth_size': [depth.width,
depth.height],
                'color_info': message_to_ordereddict(infos['color']) if 'color' in infos else
None,
                'depth_info': message_to_ordereddict(infos['depth']) if 'depth' in infos else
None,
                'registration_verified': False, 'depth_scale_m': None,
                'motion_authorized': False,
            }
            (folder/'meta.json').write_text(json.dumps(meta, indent=2))
            print('SAVED', folder, 'dt_ms=', round(dt/1e6, 2), flush=True)
            saved += 1
            last_rgb, last_depth = stamp(color), stamp(depth)
            next_save = time.monotonic() + 1.
```

```
        if saved != args.count:
            raise TimeoutError('Incomplete capture; inspect topics, QoS and clock')
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == '__main__':
    main()
PY
python3 -m py_compile "$BOTELLA_ROOT/tools/capture_rgbd.py"
```

`max-dt-ms=50` es un valor inicial para imágenes estáticas, no un permiso para controlar un objetivo móvil. No ampliarlo para ocultar un problema de tiempos. No convertir la profundidad a metros hasta verificar su escala.

6.2 Crear un lanzador de copia/captura — PC

```
cat > "$BOTELLA_ROOT/tools/capturar.sh" <<'SH'
#!/usr/bin/env bash
set -Eeuo pipefail
: "${BOTELLA_ROOT:?Cargar session.env}"
: "${BOTELLA_ROS_VISION:?Redescubrir y exportar el contenedor ROS de Vision}"
id="${1:?Uso: bash capturar.sh ID_ESCENA}"
[[ "$id" =~ ^[a-zA-Z0-9_-]+$ ]] || exit 2
remote_dir="/home/walker/botella_plan/$id"
container_dir="/tmp/botella_$id"
local_dir="$BOTELLA_ROOT/raw/$id"
[[ ! -e "$local_dir" ]] || { echo 'La escena local ya existe'; exit 2; }
ssh walker@192.168.11.3 "test ! -e $remote_dir && mkdir -p $remote_dir"
scp "$BOTELLA_ROOT/tools/capture_rgbd.py" "walker@192.168.11.3:$remote_dir/"
ssh walker@192.168.11.3 \
  "docker cp $remote_dir/capture_rgbd.py $BOTELLA_ROS_VISION:$container_dir.py"
ssh walker@192.168.11.3 \
  "docker exec $BOTELLA_ROS_VISION bash -lc 'source /opt/ros/humble/setup.bash; export
ROS2CLI_DISABLE_DAEMON=1; timeout 42 python3 $container_dir.py --out $container_dir --count 5 --
require-info'"
ssh walker@192.168.11.3 \
  "docker cp $BOTELLA_ROS_VISION:$container_dir/. $remote_dir/"
scp -r "walker@192.168.11.3:$remote_dir" "$local_dir"
printf 'CAPTURA_LOCAL=%s\n' "$local_dir"
SH
bash -n "$BOTELLA_ROOT/tools/capturar.sh"
```

El script usa nuevos IDs y rechaza sobrescritura. No elimina carpetas del robot. Si falla la copia, los datos parciales se conservan en las rutas indicadas. Consultar esas carpetas por SSH; no repetir el mismo ID ni dar por completa una toma parcial. Tras exportar y verificar hashes, la limpieza es una operación separada y selectiva.

Primera captura, robot quieto y botella colocada manualmente:

```
bash "$BOTELLA_ROOT/tools/capturar.sh" prueba_infraestructura_01
find "$BOTELLA_ROOT/raw/prueba_infraestructura_01" -maxdepth 2 -type f
```

Ver la imagen en el PC, por ejemplo:

```
xdg-open "$BOTELLA_ROOT/raw/prueba_infraestructura_01/sample_00/rgb.png"
```

Registrar los cinco `pair_dt_ms`, dimensiones, codificación y presencia de ambos `CameraInfo`. Si `--require-info` agota tiempo, volver al QoS de la fase 0; no retirar ese requisito para dar por validada una captura métrica.

6.3 Completar la cuadrícula

Colocar y registrar una escena cada vez. Ejemplos:

```
bash "$BOTELLA_ROOT/tools/capturar.sh" p01_r000
bash "$BOTELLA_ROOT/tools/capturar.sh" p01_r120
bash "$BOTELLA_ROOT/tools/capturar.sh" p01_r240
```

Repetir para `p02` a `p09`, moviendo **la botella con el robot sin mando físico**. No automatizar un bucle que capture 27 veces la misma escena sin recolocarla. Para negativas, usar `neg01` … `neg10`; para reservadas, `test_p01_r000`, etc.

Manifiesto sugerido, en PC:

```
cat > "$BOTELLA_ROOT/manifests/escenas.csv" <<'CSV'
scene_id,split,bottle_id,position_id,yaw_deg,lighting,background,reference_frame,reference_x_m,ref
erence_y_m,reference_z_m,notes
p01_r000,development,botella01,p01,0,normal,mesa,PENDIENTE,,,,rellenar_medidas
CSV
```

No rellenar coordenadas estimadas como si fueran verdad de referencia. Registrar la incertidumbre del instrumento y la definición física del punto medido.

7. Fase 2 — detector preentrenado COCO en el PC

7.1 Elección inicial y condiciones

Se propone **YOLO11n-seg** como línea base documentada: incluye segmentación y modelos preentrenados COCO [R1, R2, R3]. La máscara permitirá excluir mejor el fondo que un rectángulo 2D. No hace falta descargar el conjunto COCO completo.

La elección es una propuesta de desarrollo, no un detector instalado ni una selección irrevocable. Las condiciones de licencia de software/pesos deben revisarse antes de distribuir la aplicación; las páginas del proveedor enlazan AGPL y opciones comerciales [R2].

Se ejecutará primero por CPU y sobre archivos, sin acceso ROS ni red del robot. No instalar PyTorch/YOLO en los contenedores Motion/Vision de control.

7.2 Crear una imagen Docker local — PC, no robot

Requiere Docker ya operativo en el PC. Si no está instalado o no hay permisos, resolver su instalación con el administrador; no ejecutar estos pasos en Motion.

```
docker version
df -h "$BOTELLA_ROOT"
cat > "$BOTELLA_ROOT/tools/Dockerfile.vision" <<'DOCKER'
FROM python:3.11-slim-bookworm
RUN apt-get update && apt-get install -y --no-install-recommends \
    libglib2.0-0 libgl1 && rm -rf /var/lib/apt/lists/*
RUN python -m pip install --no-cache-dir torch torchvision \
    --index-url https://download.pytorch.org/whl/cpu
RUN python -m pip install --no-cache-dir ultralytics
WORKDIR /work
DOCKER
docker build -f "$BOTELLA_ROOT/tools/Dockerfile.vision" \
  -t botella-vision:lab-20261015 "$BOTELLA_ROOT/tools"
docker image inspect botella-vision:lab-20261015 \
  > "$BOTELLA_ROOT/manifests/docker-image.json"
docker run --rm botella-vision:lab-20261015 python -m pip freeze \
  > "$BOTELLA_ROOT/manifests/python-packages.lock.txt"
```

Las descargas pueden ser grandes. El primer build resuelve versiones del momento; el ID de imagen y `pip freeze` son parte del registro reproducible. Guardar/exportar esa imagen si se necesita reconstrucción exacta, o elaborar una receta con versiones y base por digest después de validar la primera. No afirmar que un tag flotante reproduce el mismo entorno indefinidamente.

Descargar/verificar el modelo dentro de la carpeta de datos:

```
docker run --rm --user "$(id -u):$(id -g)" \
  -e HOME=/tmp -e YOLO_CONFIG_DIR=/work/results/yolo-config \
  -v "$BOTELLA_ROOT:/work" -w /work/models \
  botella-vision:lab-20261015 python -c \
  'from ultralytics import YOLO; m=YOLO("yolo11n-seg.pt"); print(m.names)'
sha256sum "$BOTELLA_ROOT/models/yolo11n-seg.pt" \
  > "$BOTELLA_ROOT/manifests/model.sha256"
```

Usar únicamente pesos de procedencia oficial/confiable. Esa ejecución descarga el modelo si no existe; después se puede inferir con `--network none`.

7.3 Programa de inferencia 2D — PC

```
cat > "$BOTELLA_ROOT/tools/inferir.py" <<'PY'
import argparse
import json
from pathlib import Path
import time
import cv2
import numpy as np
from ultralytics import YOLO

p = argparse.ArgumentParser()
p.add_argument('--input', type=Path, required=True)
p.add_argument('--output', type=Path, required=True)
p.add_argument('--pattern', default='rgb.png')
p.add_argument('--model', default='/work/models/yolo11n-seg.pt')
p.add_argument('--conf', type=float, default=.25)
a = p.parse_args()
a.output.mkdir(parents=True, exist_ok=False)
model = YOLO(a.model)
names = model.names.items() if isinstance(model.names, dict) else enumerate(model.names)
classes = [i for i, name in names if name.lower() == 'bottle']
if not classes:
    raise RuntimeError('Model has no bottle class')
files = sorted(a.input.rglob(a.pattern))
if not files:
    raise RuntimeError('No input images')
for image_path in files:
    image = cv2.imread(str(image_path))
    if image is None:
        raise RuntimeError('Unreadable image: ' + str(image_path))
    start = time.perf_counter()
    result = model.predict(image, classes=classes, conf=a.conf, device='cpu',
                           retina_masks=True, verbose=False)[0]
    relative = image_path.relative_to(a.input)
    folder = a.output / relative.parent
    folder.mkdir(parents=True, exist_ok=True)
    detections = []
    for i, box in enumerate(result.boxes):
        item = {'class': 'bottle', 'confidence': float(box.conf[0]),
                'bbox_xyxy': box.xyxy[0].cpu().tolist(), 'mask': None}
        if result.masks is not None:
            polygon = np.rint(result.masks.xy[i]).astype(np.int32)
            mask = np.zeros(image.shape[:2], dtype=np.uint8)
            if len(polygon) >= 3:
                cv2.fillPoly(mask, [polygon], 255)
            mask_name = 'mask_%02d.png' % i
            cv2.imwrite(str(folder/mask_name), mask)
            item['mask'] = mask_name
        detections.append(item)
    report = {'source': str(relative), 'latency_s': time.perf_counter()-start,
              'detections': detections, 'unique_candidate': len(detections) == 1,
              'motion_authorized': False}
    (folder/'detections.json').write_text(json.dumps(report, indent=2))
    cv2.imwrite(str(folder/'annotated.png'), result.plot())
    print(str(relative), len(detections), flush=True)
PY
```

Ejecutar sobre la primera escena. Elegir una carpeta de salida nueva:

```
docker run --rm --network none --user "$(id -u):$(id -g)" \
  -e HOME=/tmp -e YOLO_CONFIG_DIR=/work/results/yolo-config \
  -v "$BOTELLA_ROOT:/work" botella-vision:lab-20261015 \
  python /work/tools/inferir.py \
  --input /work/raw/prueba_infraestructura_01 \
  --output /work/results/inferencia_01
xdg-open "$BOTELLA_ROOT/results/inferencia_01/sample_00/annotated.png"
```

Verificar a simple vista que la máscara cubre **la botella real**, no el fondo, la mano o una etiqueta aislada. No interpretar una confianza alta como error métrico pequeño o permiso de agarre.

7.4 Decisión después de la primera prueba

Resultado	Acción siguiente
Una botella correcta y máscara útil	Evaluar las 27 configuraciones de desarrollo
Ninguna detección	Comprobar RGB/BGR, tamaño visible, iluminación, desenfoque y clase del modelo
Detección intermitente	Revisar resolución, oclusión, brillo y confianza con datos de desarrollo
Varias botellas candidatas	Delimitar zona o pedir selección explícita; no elegir automáticamente otra
Detecta botella pero la máscara incluye fondo	Probar segmentador de mayor capacidad o anotaciones propias
Detecta correctamente, pero profundidad falla	Mantener el detector y resolver la fase métrica; más fotos no corrigen una mala escala
PC demasiado lento	Medir latencia, probar menor resolución o GPU compatible en el PC; no cargar Motion
Falla sistemáticamente con la botella concreta	Ir a fase 3, entrenamiento específico

Para cambiar confianza, generar una salida nueva, por ejemplo `--conf 0.4`. No bajar confianza hasta obtener una detección cualquiera y declararla válida. Comparar anotaciones reales, falsos positivos y casos perdidos.

7.5 Criterios iniciales de evaluación visual

Objetivos de laboratorio propuestos, no certificación:

- Una candidata correcta en al menos 26 de las 27 configuraciones de desarrollo.
- Ninguna candidata aceptada en las 10 escenas negativas iniciales.
- Evaluación separada en las 12 configuraciones reservadas, sin reajustar parámetros con ellas.
- Registrar también los fallos por configuración: cinco frames casi iguales no sustituyen escenas distintas.
- Si falla una posición que se pretende usar físicamente, esa posición se excluye hasta resolverla.

La máscara se evaluará frente a anotación manual. Para decidir «suficiente para medir», importa especialmente que no invada fondo/mesa en la zona de agarre. La aceptación visual no sustituye G3–G7.

8. Fase 3 condicional — entrenamiento si el modelo preentrenado no basta

No entrenar antes de haber descartado fallos de captura, color, calibración o selección de clase. Si el modelo ya reconoce bien, saltar esta fase.

8.1 Datos y partición

1. Ampliar escenas diferentes: iluminación, distancia, etiqueta, giro, fondo y oclusiones relevantes.
2. Anotar contornos de la botella; excluir tapa/etiqueta solo si se define otra clase deliberadamente.
3. Conservar imágenes negativas sin botella.
4. Separar entrenamiento, validación y prueba **por sesión/configuración**, no por frames consecutivos.
5. Mantener las posiciones reservadas fuera del ajuste y del entrenamiento.

No hay un número universal suficiente. Para un ajuste piloto acotado pueden recopilarse unos cientos de imágenes variadas y evaluar la curva de aprendizaje; la guía general del proveedor recomienda conjuntos considerablemente mayores, con 1500 o más imágenes por clase como orientación [R4]. No tratar ni 135 ni 1500 como garantía de agarre. Añadir datos dirigidos a fallos comprobados.

8.2 Anotación y formato

Puede usarse una herramienta de anotación elegida por el equipo. Por ejemplo, `labelme` en un entorno separado **del PC**:

```
python3 -m venv "$BOTELLA_ROOT/venv-labels"
source "$BOTELLA_ROOT/venv-labels/bin/activate"
python --version
# Python 3.12 o superior:
python -m pip install labelme
# Si el PC usa Python 3.10/3.11, usar en su lugar: python -m pip install 'labelme<7'
labelme "$BOTELLA_ROOT/dataset"
```

Abrir las imágenes de una partición, dibujar polígonos con etiqueta `bottle` y guardar sus JSON. Revisar manualmente oclusiones y ejemplos difíciles. `labelme` requiere escritorio y dependencias Qt; si falla, usar la herramienta alternativa del equipo, sin instalar una GUI en el robot. El comando se propone, no se ha instalado/verificado esta herramienta al redactar [R9].

Exportar al formato de **segmentación YOLO**, no solo bounding boxes: cada línea es `clase x1 y1 x2 y2 ...`, coordenadas normalizadas. Para un único tipo de objeto la clase será **0**, aunque `bottle` sea 39 en el COCO completo. La conversión JSON→YOLO y su verificación forman parte del trabajo de etiquetado; no mezclar polígonos sin validación ni entrenar con las etiquetas automáticas sin corregirlas.

Estructura final, dentro de `BOTELLA_ROOT/dataset`:

```
images/train/  images/val/  images/test/
labels/train/  labels/val/  labels/test/
botella.yaml
```

Conservar un manifiesto de qué escenas quedaron en cada partición. Las negativas llevan un archivo de etiquetas vacío. Comprobar correspondencia de nombres entre imagen y etiqueta y que ningún polígono tenga menos de tres puntos.

```
cat > "$BOTELLA_ROOT/dataset/botella.yaml" <<'YAML'
path: /work/dataset
train: images/train
val: images/val
test: images/test
names:
  0: bottle
YAML
```

8.3 Entrenar y evaluar — PC

Solo después de verificar etiquetas:

```
docker run --rm --user "$(id -u):$(id -g)" \
  -e HOME=/tmp -e YOLO_CONFIG_DIR=/work/results/yolo-config \
  -v "$BOTELLA_ROOT:/work" botella-vision:lab-20261015 \
  yolo segment train model=/work/models/yolo11n-seg.pt \
  data=/work/dataset/botella.yaml epochs=50 imgsz=640 batch=4 device=cpu \
  workers=0 project=/work/results/train name=botella_v1
```

Son parámetros de una primera ejecución, no una receta optimizada. CPU puede ser lenta; antes de entrenamientos largos, medir una ejecución corta. GPU será otra configuración documentada del PC, con versiones/imagen apropiadas.

```
docker run --rm --network none --user "$(id -u):$(id -g)" \
  -e HOME=/tmp -e YOLO_CONFIG_DIR=/work/results/yolo-config \
  -v "$BOTELLA_ROOT:/work" botella-vision:lab-20261015 \
  yolo segment val \
  model=/work/results/train/botella_v1/weights/best.pt \
  data=/work/dataset/botella.yaml split=test device=cpu \
  project=/work/results/evaluation name=test_v1
```

Si mejora en desarrollo y en prueba reservada: congelar pesos/hash y volver a G2. **Si solo mejora en entrenamiento:** revisar sobreajuste, diversidad y particiones. **Si sigue fallando en un subconjunto:** ampliar precisamente ese subconjunto, revisar etiquetas o cambiar la estrategia. No conectar el movimiento para compensar un detector sin validar.

9. Fase 4 — profundidad, registro y posición 3D

9.1 Escala y calidad de profundidad

`16UC1` significa entero sin signo de 16 bits, **no prueba por sí solo que cada unidad equivalga a un milímetro**. Comprobar la escala con documentación de la cámara/controlador y un blanco opaco a distancias conocidas. Guardar factor e incertidumbre, por ejemplo en `calibration/depth_scale.json`.

Para inspeccionar una captura sin atribuir aún metros:

```
docker run --rm --network none -v "$BOTELLA_ROOT:/work" \
  botella-vision:lab-20261015 python -c \
  'import numpy as np;
a=np.load("/work/raw/prueba_infraestructura_01/sample_00/depth_native.npy"); v=a[np.isfinite(a)&
(a>0)]; print(a.shape,a.dtype); print(np.percentile(v,[5,50,95]) if v.size else
"SIN_PROFUNDIDAD_VALIDA")'
```

El cero y los valores no finitos se rechazan. Los valores máximos/sentinelas específicos del sensor se deben identificar y excluir también. Comparar varias distancias y zonas de la imagen; no fijar escala a partir de una única botella.

9.2 Alineación RGB-D y distorsión

Antes de consultar profundidad en un píxel de la máscara:

1. Confirmar resolución y `frame_id` de color, profundidad y ambos `CameraInfo`.
2. Confirmar si el driver entrega profundidad registrada al RGB, y en qué rejilla: cruda o rectificada.
3. Leer `K`, `D`, `R` y `P` de esta unidad y esta resolución.
4. Comprobar con un blanco opaco y bordes conocidos la coincidencia de color/profundidad en toda la zona de trabajo.
5. Si hace falta, rectificar/registrar usando intrínsecos y extrínsecos reales; volver a comprobar.

En la lectura de esta unidad, color tiene `D` no nulo y profundidad tiene `D=0`, aunque anuncien el mismo frame y `K`. **No basta con igual tamaño o igual frame para indexar ambas imágenes directamente.** Determinar el contrato del driver.

Una receta de rectificación, cuando se haya confirmado que la profundidad está en la rejilla rectificada del mismo marco óptico, es:

```
# Fragmento de cálculo offline, no orden al robot.
# K_rgb, D_rgb, R_rgb y P_depth proceden de los CameraInfo guardados.
map_x, map_y = cv2.initUndistortRectifyMap(
    K_rgb, D_rgb, R_rgb, P_depth[:, :3], (width, height), cv2.CV_32FC1)
rgb_rectified = cv2.remap(rgb_raw, map_x, map_y, cv2.INTER_LINEAR)
```

No aplicar este fragmento sin confirmar la premisa. Si los marcos ópticos son distintos, hacen falta extrínsecos y reproyección de puntos, incluyendo oclusiones. No corregir un desplazamiento mediante un offset 2D elegido a ojo.

Después de rectificar, ejecutar la segmentación **sobre esa imagen rectificada**. La máscara y la profundidad deben referirse a la misma rejilla. No mezclar una máscara de RGB crudo con profundidad rectificada.

9.3 Primer cálculo: punto superficial, no centro ni pose de agarre

Con correspondencia RGB-D validada, escala conocida y máscara fiable:

- Erosionar la máscara para reducir contaminación del borde.
- Seleccionar una región del cuerpo de la botella, evitando tapa, cuello y mesa.
- Rechazar valores inválidos y comprobar cuántos quedan.
- Estimar un punto de superficie y su dispersión.

Para un píxel rectificado `(u,v)` con profundidad válida `Z` y matriz de esa misma rejilla:

```
X = (u - cx) * Z / fx
Y = (v - cy) * Z / fy
Z = profundidad_en_metros
```

El código de desarrollo debe producir `point_camera.json` con frame, timestamp, calibración/hash, escala, calidad y `motion_authorized=false`. Conservar los puntos usados para verificar visualmente la selección.

La mediana de profundidad de la máscara sirve para un diagnóstico, pero no convierte automáticamente el centro del bounding box en un punto 3D correcto: ese píxel puede ser fondo, el borde u otro objeto. Se necesita un píxel real validado o una estimación sobre la nube del objeto.

El punto visible está en la superficie de la botella, no en su eje. Para un marco de agarre hace falta geometría adicional: cilindro/forma conocida, diámetro medido, estimación de eje y altura de agarre. No colocar la palma en ese punto superficial como si fuera el centro de la botella.

9.4 Casos de fallo de profundidad

Caso	Tratamiento
Muchos ceros/sentinelas	Revisar rango, exposición y propiedades del objeto; no rellenar huecos para autorizar agarre
Profundidad corresponde a mesa/fondo	Revisar transparencia y máscara; rechazar geometría
Botella transparente o brillante	Probar primero botella opaca o una funda opaca como ensayo controlado; documentar que cambia el objeto
RGB correcto pero profundidad desalineada	Resolver registro/extrínsecos; no entrenar detector para compensarlo
Poca superficie visible	Cambiar disposición/cámara mediante un procedimiento físico supervisado o limitar la tarea
Dispersión elevada	Rechazar observación; revisar causa, no promediar indefinidamente

Las dificultades con transparentes están documentadas por investigación de sensores RGB-D [R5]. ClearGrasp/KeyPose son posibles líneas de investigación, **no paquetes instalados ni soluciones validadas en este robot**.

Si el puesto permanece rígidamente calibrado, una alternativa de desarrollo es usar el plano de mesa y la geometría conocida de la botella. Esa alternativa sigue necesitando validación métrica y detección de cambios del puesto.

10. Fase 5 — coordenadas del robot y calibración del TCP

10.1 Convenciones que deben fijarse

- Usar metros y radianes internamente; declarar unidades en cada archivo.
- Distinguir marco óptico, `base_link`, marco de mesa, marco de botella y TCP.
- Mantener dirección explícita de cada transformación.
- Usar cuaterniones normalizados y declarar orden `x,y,z,w`.
- La transformación de cámara debe corresponder al momento de la imagen.
- Si la cámara se mueve con cabeza/torso/elevador, registrar esos estados y TF.
- No usar pose del mapa como sustituto de cámara→base ni de calibración de la mano.

Definimos `T_A_B` como la matriz que transforma coordenadas de B a A:

```
p_base = T_base_camera · p_camera
T_base_tcp_grasp = T_base_bottle · T_bottle_tcp_grasp
```

10.2 Inspeccionar TF — LECTURA

Dentro del contenedor ROS de Vision, con setup y daemon desactivado:

```
timeout 8 ros2 topic info -v /tf
timeout 8 ros2 topic info -v /tf_static
timeout 8 ros2 run tf2_ros tf2_echo base_link waist_front_rgbd_color_optical_frame
```

Ese comando mira la transformación disponible en vivo. No aporta por sí solo la TF exacta de una imagen histórica. Un timeout puede ser normal al limitar una herramienta continua; comprobar si imprimió una transformación válida.

Para identificar frames, guardar una captura pasiva corta:

```
ros2 bag record -s sqlite3 -o /tmp/botella_tf_01 /tf /tf_static
```

Detener con Ctrl+C. Copiar fuera del contenedor y conservar junto a la campaña, como en la fase 1. No ejecutar `ros2 bag play` sobre una captura completa de órdenes de manos/brazos para intentar implementar esta tarea.

Si `tf2_ros` no está instalado, documentarlo y usar la herramienta de lectura nativa equivalente revisada. No instalar un stack de planificación o publicar un `static_transform_publisher` inventado para hacer aparecer un frame faltante.

10.3 Calibración cámara–base

Comparar puntos físicos conocidos, distribuidos por la zona, con la proyección al marco de base. Hacen falta referencias cuya ubicación en base sea conocida, no solo distancias en píxeles. Usar un blanco calibrado o el procedimiento del proveedor. Si la extrínseca existente no pasa la prueba, realizar una calibración métrica y conservar originales, versión, incertidumbre y reversión.

Objetivo inicial de laboratorio propuesto: errores de localización de orden milimétrico y, como referencia de trabajo, alrededor de 10 mm o menos en el percentil 95. **10 mm no es un límite universal seguro**. El error total de percepción, calibración y seguimiento debe ser menor que la holgura real de la aproximación. Si la mano/botella exige más precisión, el criterio se endurece.

No pasar a movimiento por cumplir un promedio si una esquina del puesto falla.

10.4 TCP y geometría de la mano v4

El responsable técnico debe definir y verificar:

- Brazo elegido y su cadena cinemática real.
- Frame de la palma y TCP de trabajo para ese agarre.
- Geometría de colisión de mano y dedos, además del brazo.
- Transformación palma/TCP, altura de agarre y orientación de aproximación.
- Límites articulares/velocidades y postura de la otra mano.

No usar el TCP de las abrazaderas. La instalación de una mano v4 y la publicación de siete articulaciones no certifican automáticamente una calibración de TCP.

Resultado: `calibration/tcp_v4_<lado>.json` y un informe de medición. Si se usa un modelo CAD o URDF, guardar versión/hash y distinguir malla visual de colisión.

11. Fase 6 — adaptación del movimiento y planificación

11.1 Qué se conserva de la demostración PICO

Conservarla como referencia del gesto: lado de aproximación, orientación de la palma, altura de agarre y fases del movimiento. Obtener del estado articular y la cinemática validada el marco del TCP durante esa demostración.

No trasladar todas las articulaciones del `.motion` sumando un offset XYZ. Los ángulos articulares no son coordenadas cartesianas. La solución debe construir poses del TCP relativas a la botella y resolver cinemática inversa.

11.2 Waypoints relativos a la botella

Definir en el marco de la botella:

1. `observe`: postura desde la que la cámara ve la zona sin oclusión de la mano.
2. `pregrasp`: mano abierta a una distancia revisada del cuerpo de la botella.
3. `grasp`: palma orientada y dedos rodeando la zona elegida.
4. `lift_check`: elevación pequeña, definida tras revisar botella, mesa y agarre.
5. `transport`: recorrido hacia B conservando orientación y espacio libre.
6. `place`: apoyo de la botella, confirmación de descarga, apertura y retirada.

Distancias y orientaciones se calcularán/revisarán para la botella y mano reales; no se fijan aquí offsets universales ni se usan números arbitrarios como comandos. La pose superficial de la fase 4 no es directamente `grasp`.

11.3 Resolución y admisión

El planificador debe comprobar la trayectoria completa, no solo el destino:

- IK con configuración inicial medida, sin saltos entre soluciones.
- Límites de todas las articulaciones y distancias a esos límites.
- Colisión con torso, otra mano, mesa, botella y objetos del puesto.
- Márgenes e incertidumbre del objeto, TCP y flexión.
- Velocidad/aceleración, interpolación y seguimiento del controlador.
- Aproximación y retirada; no solo la fase central.
- Chasis y torso inmóviles según el alcance aprobado del MVP.
- Parada/cancelación verificable y conservación del objeto ante una interrupción.

Una solución IK no demuestra ausencia de colisión. Un destino visualmente alcanzable no demuestra que lo sea con la orientación y la mano instaladas.

11.4 Comandos disponibles para descubrir el SDK — LECTURA en Motion

```
ssh -t walker@192.168.11.2 \
  "docker exec -it $BOTELLA_APP_MOTION bash"
```

Después de entrar, ejecutar dentro del contenedor:

```
source /opt/walker/setup.bash
export ROS2CLI_DISABLE_DAEMON=1
timeout 8 rosa action info /mc/manipulation/action
timeout 8 rosa action info /mc/action/outline_sdk_play
timeout 8 rosa service list -t
```

Para esta versión se identificó este servicio de lectura:

```
timeout 8 rosa service call /mc/controller_manager/list_controllers \
  rosa_control_msgs/srv/ListControllers '{}'
```

Los binarios `robot_kinematics_sdk_test` y `robot_motion_sdk_move_upper_body` existen en la instalación inspeccionada. Su existencia no constituye un contrato validado para enviar la pose de una botella. No ejecutar demos de movimiento ni suponer opciones de CLI sin revisar su implementación/contrato.

11.5 Componentes que aún deben desarrollarse

No existe todavía un comando validado que conecte detector de botella con agarre autónomo en esta unidad. Esta fase requiere integración y pruebas de software; no se resuelve con una orden Linux adicional.

Entregables propuestos, aún no implementados por este documento:

Componente	Entrada	Salida/obligación
Observador de botella	RGB-D, calibración, selección	Objeto único, pose, incertidumbre, frame, edad, ID de observación
Generador de agarre	Pose de objeto, TCP, modelo de botella	Waypoints cartesianos revisables
Adaptador IK/planificador	Waypoints, estado y escena actual	Trayectoria articular admitida o motivo de rechazo
Ejecutor de tarea	Plan admitido, estado fresco, autorización física	Máquina de estados, cancelación y resultado por fase
Verificador de sujeción	Señales de mano y visión	Confirmación, fallo o estado indeterminado

El adaptador debe usar la interfaz nativa y los controladores apropiados después de validar el contrato. No conectar el publicador VLA en shadow ni reproducir órdenes EtherCAT crudas como atajo.

Las interfaces futuras deberían separar `observe`, `plan`, `check` y `run`; `observe/plan/check` no publicarán movimiento y el ejecutor arrancará bloqueado por defecto. Son nombres de diseño, **no comandos existentes para copiar**.

11.6 Pruebas de software antes de movimiento

Preparar tests con estos casos:

- Botella no detectada, varias candidatas o candidata fuera de zona.
- Profundidad inválida, unidad incorrecta o alineación incompatible.
- TF ausente, equivocada, antigua o de otro frame.
- Objetivo que cambia mientras se calcula el plan.
- Postura inicial distinta, dato articular antiguo o incompleto.
- IK sin solución, solución discontinua, límite o colisión intermedia.
- Botella desaparece o deja de estar quieta.
- Señal de mano insuficiente, esfuerzo saturado o fallo de seguimiento.

- Paro, pérdida de enlace, cancelación y reconexión.
- Reanudación con botella sujeta frente a apoyada: deben ser estados distintos.

Guardar planes y visualizar la escena/TCP antes de habilitar ejecución. Usar simulación solo si robot/efector/colisiones están bien modelados; no inferir seguridad física porque la animación se vea correcta.

12. Fase 7 — prensión, fuerza y primera prueba física

12.1 Lo conocido sobre el perfil actual

El YAML de teleop consultado usa `hand_ctrl_mode: cur`, límites `effort_ub=1.1` y una configuración de cierre. En la toma 05 se observaron modos de mano 5/7 y cambios en `torque` izquierdo. Las unidades/semántica exactas deben contrastarse con el proveedor/controlador antes de usarlas como medida de fuerza de contacto.

No afirmar que 1.1 sean newtons, amperios o un porcentaje, ni que sea la fuerza máxima física. Tampoco afirmar regulación automática de fuerza mínima.

La reproducción geométrica que se comprobó usa posiciones medidas de dedos; **no equivale a un control de agarre con objeto ni a repetir la corriente**.

12.2 Información necesaria antes de agarrar

Solicitar/confirmar límites de carga y presión para las manos y el objeto, comportamiento de corriente/posición, modo de mantener agarre, señales disponibles y política de fallo. Revisar si existen sensores táctiles, corriente por dedo, esfuerzo estimado y detección de contacto/deslizamiento realmente utilizable.

Un aumento de corriente aislado puede indicar contacto, atasco o colisión; no basta para afirmar que la botella está sujeta. Combinar señales de cierre, geometría esperada y comprobación visual. No aumentar límites a ciegas si cae.

12.3 Secuencia física gradual

1. Con G0–G5 cerradas, ensayar trayectoria de acercamiento con la botella retirada.
2. Mantener pausas entre fases y amplitud/velocidad reducidas mediante el planificador admitido.
3. Verificar la postura de la mano en el punto de preagarre, sin contacto.
4. Colocar botella ligera y no frágil; repetir aproximación y cerrar de forma supervisada.
5. Comprobar sujeción antes de una elevación pequeña, sobre una zona despejada.
6. Si la comprobación es satisfactoria, ejecutar apoyo/depósito cercano y apertura.
7. Ampliar recorrido a B únicamente después de validar la retención.

Cerrar dedos, levantar y transportar son acciones físicas nuevas. No ejecutar un `.motion` de mano vacía sobre una botella como sustituto de estas validaciones.

12.4 Máquina de estados y errores

```
IDLE → OBSERVING → TARGET_VALID → PLAN_VALID
  → PREGRASP → APPROACH → CLOSING → HOLD_CHECK
  → LIFT_CHECK → TRANSPORT → PLACING → RELEASE_CHECK → RETREAT → DONE
```

Ante fallo: registrar la fase, detener/rechazar la acción según el contrato y clasificar el objeto como **libre / sujeta / apoyada / indeterminada**.

- Antes del contacto: cancelar y volver a observar solo después de reposo confirmado.

- Durante cierre: no elevar si no se conoce el estado del objeto.
- Durante transporte: no abrir automáticamente como respuesta genérica al error.
- Durante apoyo: confirmar apoyo antes de liberar.
- Ante estado indeterminado: recuperación específica supervisada; sin HOME o reinicio de ciclo automático.

13. Fase 8 — demostraciones y grabación completa

13.1 Movimiento y dedos — herramientas existentes

En una terminal **del PC**, desde el repo:

```
bash scripts/teleoperation/record_pico_v4_complete.sh --check
bash scripts/teleoperation/record_pico_v4_complete.sh --record botella_demo_01
```

Esperar las suscripciones. En otra terminal **del PC**, solo tras condiciones físicas actuales y perfil v4 cargado:

```
PICO_TELEOP_CAMERA=off CRUZR_IFACE=eno1 CRUZR_PC_IP=192.168.11.250 \
  bash scripts/teleoperation/probar_pico_v4.sh --check

PICO_TELEOP_CAMERA=off CRUZR_IFACE=eno1 CRUZR_PC_IP=192.168.11.250 \
  bash scripts/teleoperation/probar_pico_v4.sh --teleoperate
```

FÍSICO: seguir la confirmación del lanzador; Y izquierdo una vez solo al aviso. Head+Controllers/Send data en PICO. No usar clicks de joystick ni otros mandos simultáneos. El perfil debe confirmar mano v4 y cintura/elevador fuera de control. Si aparece `ACTIVE_TASK_ERROR=cruzr/home`, no se ha cargado la tarea PICO v4: seguir el procedimiento supervisado de la ficha v4. No iniciar otro mando ni cambiar a clamp para pasar la comprobación.

Ctrl+C en el grabador guarda; **no detiene la teleoperación**. El lanzador teleop tiene su propia parada/ventana. La cámara PICO `off` afecta al vídeo hacia el visor, no implica apagar las cámaras ROS del robot.

13.2 Añadir visión para un futuro dataset

El grabador completo de movimientos no incluye automáticamente RGB-D. Para correlacionar demostraciones con la botella, grabar también, en Vision:

```
source /opt/ros/humble/setup.bash
export ROS2CLI_DISABLE_DAEMON=1
ros2 bag record -s sqlite3 -o /tmp/botella_vision_demo_01 \
  /sensor/camera/waist_front_rgbd/color/raw \
  /sensor/camera/waist_front_rgbd/depth/raw \
  /sensor/camera/waist_front_rgbd/color/info \
  /sensor/camera/waist_front_rgbd/depth/info /tf /tf_static
```

Antes de una toma útil, comprobar los QoS de todos esos canales. Rosbag adapta QoS durante discovery [R6]; si hay incompatibilidad, usar un archivo explícito por topic y verificar recuentos, incluyendo calibración/TF latcheadas. No aplicar `VOLATILE` indiscriminadamente a `CameraInfo` o `tf_static`.

Si faltan calibración o TF latcheadas, este archivo permite solicitar el QoS observado para los canales de calibración y el habitual de TF estática. Crearlo **dentro del contenedor ROS de Vision**, después de contrastar los publishers:

```
cat > /tmp/botella_qos.yaml <<'YAML'
/sensor/camera/waist_front_rgbd/color/info:
  reliability: reliable
  durability: transient_local
  history: keep_last
  depth: 1
/sensor/camera/waist_front_rgbd/depth/info:
  reliability: reliable
  durability: transient_local
  history: keep_last
  depth: 1
/tf_static:
  reliability: reliable
  durability: transient_local
  history: keep_last
  depth: 100
YAML
ros2 bag record -s sqlite3 -o /tmp/botella_vision_demo_02 \
  --qos-profile-overrides-path /tmp/botella_qos.yaml \
  /sensor/camera/waist_front_rgbd/color/raw \
  /sensor/camera/waist_front_rgbd/depth/raw \
  /sensor/camera/waist_front_rgbd/color/info \
  /sensor/camera/waist_front_rgbd/depth/info /tf /tf_static
```

Tras Ctrl+C, comprobar `ros2 bag info /tmp/botella_vision_demo_02`. Si algún canal necesario tiene cero mensajes, la toma no cumple el contrato. La receta no modifica el QoS del publicador ni el servicio de cámaras.

Estas imágenes SHM incluyen buffers de capacidad fija: estimar volumen con una captura corta antes de una sesión larga. No llenar el disco de Vision. Conservar metadata y archivos originales; exportar/copiar fuera del robot.

Usar timestamps originales para alinear manos, brazos, imágenes y TF. Motion, Vision y PC pueden tener desfases de reloj; el PC mostró diferencias respecto a los hosts del robot en esta intervención. Comparar relojes y estabilidad:

```
date -u -Is
ssh walker@192.168.11.2 'date -u -Is'
ssh walker@192.168.11.3 'date -u -Is'
```

Estas tres consultas son diagnósticas, no calibran un reloj con precisión submilisegundo. No sincronizar por nombre del archivo o por el «instante de pulsar Enter», ni ajustar el reloj del robot mientras controla una tarea.

Para entrenar un VLA más adelante serán necesarios episodios completos, modalidades/acciones compatibles con v4, divisiones por sesión y nuevos gates; el checkpoint de cajas de 20D no resuelve automáticamente manos/botellas. Consultar la guía de datos VLA.

14. Fase 9 — validación del sistema completo

14.1 Tabla de ensayo

Registrar cada intento, incluidos los fallos:

```
cat > "$BOTELLA_ROOT/manifests/ensayos.csv" <<'CSV'
attempt_id,scene_id,model_sha256,calibration_id,plan_id,detected,pose_error_mm,pregrasp_ok,hold_co
nfirmed,lift_ok,deposit_ok,result,failure_phase,notes
CSV
```

Propuesta de progresión:

1. Una posición, primero en vacío y luego con botella ligera, hasta aclarar todas las incidencias.
2. Tres posiciones distintas, incluidas variaciones laterales y de profundidad.
3. Las nueve posiciones de desarrollo, sin saltar las difíciles.
4. Las cuatro posiciones reservadas y nuevas orientaciones/apariencias del alcance declarado.
5. Añadir una sola variación adicional cada vez: otra botella, iluminación, masa o destino.

El responsable técnico debe fijar antes de la campaña número de repeticiones, tasa de éxito y condiciones de rechazo. Como campaña exploratoria, tres ensayos por posición ofrecen información inicial, pero no certifican fiabilidad industrial. No seleccionar solo vídeos exitosos para valorar el resultado.

14.2 Qué significa éxito

Una captura visual no es un agarre. Un cierre de mano no es sujeción confirmada. Un levantamiento no demuestra transporte estable. El resultado A→B exige:

- Botella correcta y pose válida.
- Aproximación sin contacto no previsto.
- Sujeción verificada y ausencia de deslizamiento relevante.
- Depósito estable en B y liberación confirmada.
- Retirada y reposo final medidos.
- Registro reproducible y estados de fallo recuperables.

14.3 Si el ensayo falla

Síntoma	Primera revisión	Evitar
Se acerca al fondo	Máscara, transparencia, correspondencia RGB-D	Cambiar límites de alcance
Error de escala constante	Unidad de profundidad y escala	Multiplicar coordenadas a ojo
Error cambia según imagen	Distorsión, registro y extrínsecos	Un único offset global sin calibrar
Error cambia con postura	TF temporal, modelo cinemático y TCP	Usar la misma TF histórica
Mano llega bien, dedos no sujetan	Geometría de prensión, modo y señales de mano	Subir corriente sin contrato
Botella se deforma	Material, contacto y esfuerzo aplicado	Repetir el mismo cierre más fuerte
Botella se desliza	Contactos, masa, fricción y control de retención	Suponer que `effort` equivale a fuerza normal útil
IK no resuelve	Orientación, semilla, límites y zona	Saltarse el planificador con comandos crudos
Pérdida de tracking PICO	Estado real del visor y stream	Aceptar Working como prueba suficiente
Cae el enlace durante tarea	Estado físico y cancelación/paro según contrato	Reintento o HOME automático

15. Entregables, cierre y mantenimiento

Entregable	Se considera completo cuando
Inventario de unidad	Versiones, tipos, rutas y frames guardados, sin secretos
Dataset de evaluación	Escenas positivas/negativas/reservadas identificadas, imágenes válidas y referencia medida
Detector	Pesos/hash, entorno, métricas y fallos conservados
Calibración	Escala, registro, extrínsecos y TCP pasan verificaciones en toda la zona
Adaptación de trayectorias	IK/colisiones/continuidad revisados y tests de errores pasan
Prensión	Contrato y señales confirmados; ensayos de sujeción y fallo documentados
Ejecutor	Estados, gates, resultado, cancelación y recuperación específicos probados
Validación final	Tasa de éxito y alcance declarados con casos no usados para ajustar

Guardar checksums de artefactos, fuera de Git:

```
find "$BOTELLA_ROOT/raw" -type f -print0 \
  | sort -z | xargs -0 -r sha256sum \
  > "$BOTELLA_ROOT/manifests/raw.sha256"
sha256sum "$BOTELLA_ROOT/tools/"*.py "$BOTELLA_ROOT/tools/"*.sh \
  > "$BOTELLA_ROOT/manifests/tools.sha256"
```

Mantener el código definitivo y recetas en el repositorio; los prototipos de los heredocs deben convertirse en herramientas versionadas/revisadas antes de uso operativo. Conservar imágenes, bags y pesos fuera de Git. Documentar cualquier cambio de PC, Vision o Motion en `SYSTEM_CUSTOMIZATIONS.md`, con copia anterior, fuente, versión/hash, activación, verificación y reversión. No sustituir una adaptación compatible por una configuración antigua completa.

Al cerrar una prueba, clasificar objeto y postura. Antes de apagar, seguir el procedimiento actual de HOME/apagado cuando la trayectoria esté libre; nunca mandar HOME con botella sujeta o una mesa dentro del recorrido. Una emergencia se trata según la guía de recuperación y no como un apagado normal.

16. Orden práctico de las siguientes sesiones

1. **Sesión de infraestructura:** inventario y una captura RGB-D estática de prueba.
2. **Sesión visual:** ejecutar COCO sobre esa imagen; si funciona, cuadrícula y evaluación; si falla, diagnóstico y rama de entrenamiento.
3. **Sesión métrica:** registro RGB-D, escala, puntos de referencia y TF; sin movimiento.
4. **Sesión de geometría:** TCP de v4, marco de botella y planes en shadow; sin publicador físico.
5. **Sesión en vacío:** trayectoria pequeña admitida, supervisada, sin botella en la trayectoria.
6. **Sesión de prensión:** botella ligera, comprobación de cierre, elevación y apoyo cercano.
7. **Sesión A→B:** transporte y depósito, seguida de validación en posiciones reservadas.

El primer hito es **ver una máscara correcta de vuestra botella en una imagen real del robot**. No hace falta comenzar recogiendo cientos de demostraciones PICO ni entrenando un VLA.

17. Referencias y límites de la documentación

Documentación local, prioritaria para esta unidad:

- Fuente de verdad del proyecto.
- Personalizaciones del sistema.
- Catálogo de funcionalidades.
- PICO y manos v4: instalación, captura y replay.
- Procedimiento de grabación/reproducción.
- Datos y compatibilidad VLA.
- Recuperación tras contacto/teleop.

Fuentes externas primarias consultadas el 15-10-2026. Son capacidades de las herramientas/modelos, no ensayos realizados en Cruzr:

- **R1:** Ultralytics: COCO, clases y modelos. Incluye `bottle`.
- **R2:** Ultralytics: YOLO11, tareas, pesos y licencias.
- **R3:** Ultralytics: segmentación y máscaras.
- **R4:** Ultralytics: variedad y cantidad de datos. Orientación general, no requisito certificado para este puesto.
- **R5:** ClearGrasp, publicación primaria. Limitaciones de profundidad con transparentes.
- **R6:** ROS 2 Humble: QoS de grabación y reproducción. Véase también el README oficial de rosbag2, sección QoS. Contrastar las opciones con `ros2 bag record --help` de Humble; la rama principal puede describir opciones posteriores.
- **R7:** Ultralytics: ejecución con Docker.
- **R8:** Ultralytics: inferencia.
- **R9:** Labelme: repositorio y uso oficiales.

Límite explícito: este documento proporciona comandos de infraestructura, captura, inferencia y herramientas existentes, además de especificar la integración de agarre todavía pendiente. No inventa una CLI de agarre autónomo que no existe. G5–G8 requieren implementación, revisión y pruebas adicionales.
